

Heterogeneity in Dynare 7

Stationary Distribution and Steady State

Dynare Masterclass 2026

Normann Rion ¹

¹Dynare Team

July 9, 2026

Roadmap

- ➊ **The steady state:** definition and discretization.
- ➋ **Inputs:** .mod-file syntax for `heterogeneity_load_steady_state` and `heterogeneity_compute_steady_state`.
- ➌ **Algorithm:** time iteration, Fischer–Burmeister, forward iteration, outer Broyden.
- ➍ **Comparison with the sequence-space package:** similarities, discrepancies, pros and cons

Table of Contents

- 1 The steady state
- 2 Inputs in the .mod file
- 3 Algorithm
- 4 The sequence-space package: a comparison

The abstract framework (reminder)

As in Bhandari et al. [2023] (BBEG), a continuum of agents $i \in \mathcal{I}$, discrete time.

Red: heterogeneous. **Blue:** aggregate.

$$F(\theta_{i,t}, a_{i,t-1}, x_{i,t}, \mathbb{E}_{i,t} x_{i,t+1}, Y_t) = 0$$

$$G\left(\int y_{i,t} di, A_{t-1}, X_t, \mathbb{E}_t X_{t+1}, \mathcal{E}_t\right) = 0$$

where

- $x_{i,t}, X_t$: endogenous variables
- $a_{i,t-1} \subseteq x_{i,t-1}, A_{t-1} \subseteq X_{t-1}$: pre-determined variables
- $\theta_{i,t}$: heterogeneous exogenous variables
- $\mathcal{E}_t$: aggregate i.i.d Gaussian innovations
- $Y_t \subseteq [A_{t-1}, X_t, \mathbb{E}_t X_{t+1}, \mathcal{E}_t]$: relevant aggregate variables
- $y_{i,t} \subseteq x_{i,t}$: relevant aggregated heterogeneous variables

Recursive representation

- Aggregate state $Z = [\mathcal{E}, A, \Omega]'$, individual state (θ, a, Z) .
- Individual policies: $\bar{x}(\theta, a, Z)$, $\bar{a}(\theta, a, Z)$.
- Aggregate policies: $\bar{X}(Z)$, $\bar{\Omega}(Z)$.
- Equilibrium equations:

$$\begin{aligned}
 F(\theta, a, \bar{x}(\theta, a, Z), \mathbb{E}_{\theta'|\theta, \mathcal{E}'}[\bar{x}(\theta', \bar{a}(\theta, a, Z), Z')], \bar{Y}(Z)) &= 0, \\
 G(\int \bar{y}(\theta, a, Z) d\Omega(\theta, a), P\bar{X}(Z), \bar{X}(Z), \mathbb{E}_{\mathcal{E}'}\bar{X}(Z'), \mathcal{E}) &= 0, \\
 \bar{\Omega}(Z)(B) &= \int \mathbf{1}\{(\theta', \bar{a}(\theta, a, Z)) \in B\} d\pi(\theta' | \theta) d\Omega(\theta, a).
 \end{aligned}$$

with $\bar{Y}(Z) \subseteq [\mathcal{E}, P\bar{X}(Z), \bar{X}(Z), \mathbb{E}_{\mathcal{E}'}\bar{X}(Z')]$ the relevant aggregate slice.

Steady state

Switch off the aggregate innovation forever, $\mathcal{E} = 0$

- Aggregate variables freeze at X^*
- The cross-section settles at Ω^*
- The policy functions become time-invariant: $\bar{x}^*(\theta, a), \bar{a}^*(\theta, a)$

The fixed-point equations are

$$F(\theta, a, \bar{x}^*(\theta, a), \mathbb{E}_{\theta'|\theta}[\bar{x}^*(\theta', \bar{a}^*(\theta, a))], Y^*) = 0,$$

$$G(\int \bar{y}^* d\Omega^*, PX^*, X^*, X^*, 0) = 0,$$

$$\Omega^*(B) = \int \mathbf{1}\{(\theta', \bar{a}^*(\theta, a)) \in B\} d\pi(\theta' | \theta) d\Omega^*(\theta, a),$$

with $Y^* \subseteq [0, PX^*, X^*, X^*]$ the relevant-aggregate slice picked from X^* .

→ Three unknowns: $\bar{x}^*$ (policy), Ω^* (cross-section), X^* (aggregate vector).

Krusell–Smith at the steady state

Plug in $\mathbf{x} = [c, k]'$, $\mathbf{a} = [k]'$, $\theta = [e]'$, $\mathbf{Y} = [r^*, w^*]'$.

→ The household block becomes

$$c^*(k, e)^{-1/\eta} - \beta(1 + r^*) \mathbb{E}_{e'|e} [c^*(k^*(k, e), e')^{-1/\eta}] \geq 0 \quad \perp \quad k^*(k, e) \geq 0,$$

$$c^*(k, e) + k^*(k, e) = (1 + r^*)k + w^* \exp(e).$$

→ The aggregate block freezes at $Z^* = Z_{ss}$:

$$r^* = \alpha Z^* (K^*/L)^{\alpha-1} - \delta, \quad w^* = (1 - \alpha) Z^* (K^*/L)^\alpha,$$

$$K^* = \int k^*(k, e) d\Omega^*(k, e).$$

Discretized steady state

- Per heterogeneous shock $e \in \theta$: Markov-chain discretization
 - a grid of nodes $\theta_{[e]}$
 - a transition matrix $\pi_{[e]}$
- Per pre-determined state $k \in a$, 2 grids:
 - policy grid k^p , where time iteration runs (coarse);
 - distribution grid k^d , where forward iteration runs (dense).
- Discrete objects on the joint mesh $\theta \times a^*$ (tensor product of all grids):
 - policy array x
 - histogram Ω
- Aggregate endogenous variables X^*

Table of Contents

- 1 The steady state
- 2 Inputs in the .mod file**
- 3 Algorithm
- 4 The sequence-space package: a comparison

The 2 commands

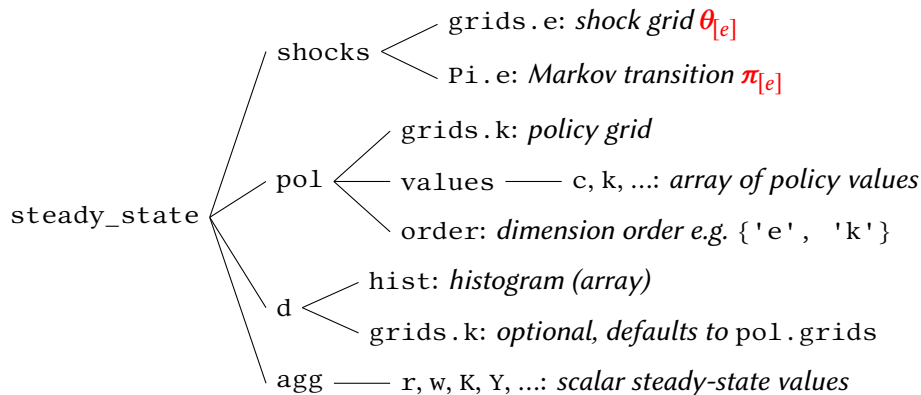
Both consume the same root MATLAB structure from

- a .mat file
- a workspace variable

```
heterogeneity_load_steady_state(filename = FILE, variable = NAME, [options...]);
heterogeneity_compute_steady_state(filename = FILE, variable = NAME, [options...]);
```

- *_load_*: read the structure, populate oo_.heterogeneity.steady_state.
- *_compute_*: read the structure as an *initial guess*, solve the steady state, populate oo_.heterogeneity.steady_state and M_.params accordingly.

The MATLAB/Octave steady-state structure for *_load_*



Options tolf:

- load checks $\|G(\sum \Omega \cdot \mathbf{y}, PX^*, X^*, X^*, 0)\|_\infty < \text{tolf}$.
- the heterogeneous block is trusted (we'll see later why).

Calibration as finding a residual map zero

Partition the inputs and the aggregate equations:

- $\beta \in \mathbb{R}^{n_\beta}$: *free parameters*, peeled off the calibrated set
→ e.g. β in Krusell–Smith.
- $H \subseteq G$: n_τ aggregate equations picked as *targets*
→ e.g. capital market clearing condition $K_t = \int k_{i,t} di$

For any β , the remaining equations have a unique steady state $(\bar{x}^*, \Omega^*, X^*)$. The *calibration residual map*

$$R : \mathbb{R}^{n_\beta} \rightarrow \mathbb{R}^{n_\tau}, \quad R(\beta) \equiv H\left(\int \bar{y}^* d\Omega^*, PX^*, X^*, X^*, 0\right)$$

returns the residual of the target equations at that steady state.

Calibration as finding a residual map zero

Partition the inputs and the aggregate equations:

- $\beta \in \mathbb{R}^{n_\beta}$: *free parameters*, peeled off the calibrated set
→ e.g. β in Krusell–Smith.
- $H \subseteq G$: n_τ aggregate equations picked as *targets*
→ e.g. capital market clearing condition $K_t = \int k_{i,t} di$

For any β , the remaining equations have a unique steady state $(\bar{x}^*, \Omega^*, X^*)$. The *calibration residual map*

$$R : \mathbb{R}^{n_\beta} \rightarrow \mathbb{R}^{n_\tau}, \quad R(\beta) \equiv H\left(\int \bar{y}^* d\Omega^*, PX^*, X^*, X^*, 0\right)$$

returns the residual of the target equations at that steady state.

The *calibration problem*: find β^* with

$$R(\beta^*) = 0, \quad n_\beta = n_\tau \text{ (squareness).}$$

The algorithm at a glance in *_compute_*

Steps 1–3 evaluate $R(\beta)$. Step 4 zeros it.

① Time iteration:

- ① solve $F = 0$ at every node of $\theta \times \mathbf{a}^*$
- ② iterate $\mathbf{x}$ to a fixed point.

② Forward iteration: push Ω through the policy-induced transition until convergence.

③ Aggregation: compute $\int \mathbf{y} d\Omega$, evaluate $H \Rightarrow R(\beta)$.

The algorithm at a glance in *_compute_*

Steps 1–3 evaluate $R(\beta)$. Step 4 zeros it.

① Time iteration:

- ① solve $F = 0$ at every node of $\theta \times \mathbf{a}^*$
- ② iterate $\mathbf{x}$ to a fixed point.

② Forward iteration: push Ω through the policy-induced transition until convergence.

③ Aggregation: compute $\int \mathbf{y} d\Omega$, evaluate $H \Rightarrow R(\beta)$.

Outer wrap when the `free_parameters` field is provided:

④ Calibration: Broyden adjusts β until $\|R(\beta)\|_\infty < \text{calibration_tol}$.

→ No `free_parameters`: steps 1–3 run once, all aggregate residuals are printed.

→ Section 3 unpacks each step; the next slide hooks the user-controllable knobs to them.

MATLAB/Octave structure for *_compute_*

Same root structure as *_load_*, with two changes:

- `d.hist` is ignored: forward iteration recomputes it.
- Optional `free_parameters` field carries β
→ one sub-struct per parameter:

```
initial_guess.free_parameters.beta.initial_guess = 0.98;
initial_guess.free_parameters.beta.lower_bound  = 0.97;
initial_guess.free_parameters.beta.upper_bound  = 0.99;
```

→ option `calibration_target_equations` = ['name1', 'name2'];
default target equations H : aggregate equations containing SUM

- No `free_parameters` field: returns $R(\beta)$ where $\beta = M_params$.

Inline vs `verbatim`: the preprocessor's view

The preprocessor treats statements differently depending on where they sit.

Statement	Inline	Inside <code>verbatim</code>
<code>delta = 0.025;</code> (parameter)	updates <code>M_.params</code>	MATLAB local only
<code>r = 0.01;</code> (declared var)	syntax error	MATLAB local
<code>rk = r + delta;</code> (free name)	parameter substitution	literal pass-through

- Parameter assignments *must* sit inline to land in `M_.params`.
- Variable assignments *must* sit in `verbatim` (pre-processor requirement).
- Anything that builds the `initial_guess` struct lives in `verbatim`.
- Chain the two: inline parameter → `verbatim` block reading it.

→ Hackish! Future ⇒ proper interface for input through a `.m` file.

Worked example: Krusell–Smith

```

delta = 0.025; eis = 1; L = 1; alpha = 0.11;
verbatim;
    r = 0.01; rk = r + delta; // declared variable, free name -> verbatim
end;
Z_ss = (rk / alpha) ^ alpha; // declared parameter -> inline
verbatim; // declared variables, initial_guess structure -> verbatim
    K = (alpha * Z_ss / rk) ^ (1/(1-alpha));
    w = (1 - alpha) * Z_ss * (K/L)^alpha;
    initial_guess.agg.r = r; initial_guess.agg.w = w;
    initial_guess.agg.K = K; initial_guess.agg.Y = Z_ss * K^alpha;
    initial_guess.free_parameters.beta.initial_guess = 0.98;
    [grid_e, ~, Pi_e] = rouwenhorst(0.966, 0.5, 3, 1e-12, 1e5);
    initial_guess.shocks.grid_e = grid_e; initial_guess.shocks.Pi_e = Pi_e;
    initial_guess.pol.grid_k = logspace(log10(.25), log10(200.25), 30) -.25;
    coh = (1+r) * initial_guess.pol.grid_k + w * grid_e;
    initial_guess.pol.values.c = 0.1 * coh; initial_guess.pol.values.k = 0.9 * coh;
    initial_guess.pol.order = {'e', 'k'};
end;
heterogeneity_compute_steady_state(variable = initial_guess);

```

Table of Contents

- 1 The steady state
- 2 Inputs in the .mod file
- 3 Algorithm**
- 4 The sequence-space package: a comparison

Time iteration (step 1)

At every node $(\theta, a) \in \boldsymbol{\theta} \times \mathbf{a}^P$ of the joint mesh, solve for the half-step policy $\mathbf{x}^{(n+1/2)}(\theta, a)$ given the previous-iteration policy $\mathbf{x}^{(n)}$

$$F(\theta, a, \mathbf{x}^{(n+1/2)}(\theta, a), \mathbb{E}_{\theta'|\theta}[\mathbf{x}^{(n)}(\theta', \bar{a}^{(n+1/2)}(\theta, a))], Y^*) = 0$$

via a trust-region method (Fischer–Burmeister for any $\perp$ clause).

- Off-node evaluation of $\mathbf{x}^{(n)}$ uses multi-linear interpolation on $\mathbf{a}^P$.
- Dampened update: $\mathbf{x}^{(n+1)} = (1 - \lambda) \mathbf{x}^{(n)} + \lambda \mathbf{x}^{(n+1/2)}$, with $\lambda = \text{time_iteration_learning_rate}$ (default 1).
- Convergence: $\|\mathbf{x}^{(n+1/2)} - \mathbf{x}^{(n)}\|_\infty < \text{time_iteration_tol}$ (default 10^{-8}).
- Early-stop guard: abort if the update norm grows for `time_iteration_early_stopping` consecutive iterations.

Fischer–Burmeister regularization

A complementarity clause $u \geq 0 \perp w \geq 0$ is replaced by one smooth scalar equation:

$$\phi_{FB}(u, w) = u + w - \sqrt{u^2 + w^2 + \varepsilon} = 0, \quad \varepsilon = 10^{-14}.$$

- The step-1 trust region algorithm sees a smooth system
- Same idiom as Dynare's 1mmcp solver (see the manual).
- Post-convergence: snap constrained coordinates to the bound when within $\sqrt{\varepsilon} \approx 10^{-7}$ of it.

Forward iteration of the distribution (step 2)

Fixed-point algorithm:

- Initialization: stationary shock marginal $\otimes$ uniform on $\mathbf{k}^d$.
- Convergence: $\|\Omega^{(n+1)} - \Omega^{(n)}\|_1 < \text{forward_tol}$ (default 10^{-10}), checked every $\text{forward_check_every}$ iterations (default 100).

On each iteration, the law of motion of Ω can be decomposed as:

- 1 **A lottery on the predetermined state** [Young, 2010]:
split the mass at each node (θ, k) between the $\mathbf{k}^d$ nodes bracketing $\bar{a}^*(\theta, k)$, by linear interpolation.
- 2 **A shock transition:**
apply the joint Markov kernel π on the shock axes.

Aggregation and outer Broyden (steps 3 & 4)

Aggregation (step 3).

- Cross-sectional integrals become finite sums on the joint mesh, $\int \bar{y}^* d\Omega^* \approx \sum \Omega \cdot y$.
- Plug into G , extract residuals on target equations $\rightarrow R(\beta) \in \mathbb{R}^{n_\tau}$.

Calibration (step 4, Broyden)

- Default target equations: aggregate equations containing SUM. Override with `calibration_target_equations = ['name1', ...]`.
- Squareness: `#free_parameters = #target equations`.
- Box-constraint step rejection on `free_parameters.*.lower_bound/upper_bound` if supplied
- Options:
 - Convergence: $\|R(\beta)\|_\infty < \text{calibration_tolf}$ (default 10^{-4})
 - Cap `calibration_max_iter` (default 50)

Table of Contents

- 1 The steady state
- 2 Inputs in the .mod file
- 3 Algorithm
- 4 The sequence-space package: a comparison**

A common problem represented differently

Package the heterogeneous block as a functional, taking a candidate current policy $\bar{x}$, a continuation policy $\bar{x}^+$, and the relevant-aggregate slice Y :

$$\mathcal{F}[\bar{x}, \bar{x}^+, Y](\theta, a) \equiv F(\theta, a, \bar{x}(\theta, a), \int \bar{x}^+(\theta', p \bar{x}(\theta, a)) d\pi(\theta' | \theta), Y).$$

- $\bar{x}, \bar{x}^+$: candidate policy functions (current period, next period)
- p : selection matrix for the predetermined states, $\bar{a} = p \bar{x}$.
- Implicit function theorem at the steady state: under invertibility of $D_{\bar{x}} \mathcal{F}(\bar{x}^*, \bar{x}^*, Y^*)$, there is a locally unique backward operator $\mathfrak{x}(\bar{x}^+, Y)$ with

$$\mathcal{F}[\mathfrak{x}(\bar{x}^+, Y), \bar{x}^+, Y] = 0.$$

- In Auclert et al. [2021], the user provides $\mathfrak{x}$ (typically, the EGM step).
- In Dynare, the user provides F .

Dynare embeds SSJ: residual geometry and trade-offs

Different residual geometries:

- SSJ: zeros its own EGM consistency check.
- Dynare: zeros per-node FOC residuals on its joint mesh.

→ `heterogeneity_load_steady_state` validates only aggregate residuals; heterogeneous block trusted.

Dynare embeds SSJ: residual geometry and trade-offs

Different residual geometries:

- SSJ: zeros its own EGM consistency check.
- Dynare: zeros per-node FOC residuals on its joint mesh.

→ `heterogeneity_load_steady_state` validates only aggregate residuals; heterogeneous block trusted.

Dynare's perks (cost: per-node Newton, slower)

- kinks in $\mathbb{R}$, smoothed by Fischer–Burmeister;
- lagged aggregates X_{t-1} in the heterogeneous block;
- more than two individual states;
- no DAG decomposition required.

Bibliography

- Adrien Auclert, Bence Bardóczy, Matthew Rognlie, and Ludwig Straub. Using the sequence-space Jacobian to solve and estimate heterogeneous-agent models. *Econometrica*, 89(5):2375–2408, 2021. doi: 10.3982/ECTA17434.
- Anmol Bhandari, Thomas Bourany, David Evans, and Mikhail Golosov. A perturbational approach for approximating heterogeneous agent models. Working Paper 31744, National Bureau of Economic Research, September 2023. URL <http://www.nber.org/papers/w31744>.
- Eric R. Young. Solving the incomplete markets model with aggregate uncertainty using the Krusell–Smith algorithm and non-stochastic simulations. *Journal of Economic Dynamics and Control*, 34(1):36–41, 2010. doi: 10.1016/j.jedc.2008.11.010.